DATE: 15/10/2026

CO1-AT1- CODING CHALLENGE

COURSE CODE: DSA0308

COURSE NAME: NATURAL LANGUAGE PROCESSING

REG NO: 192324235 **NAME:** JOTHIKA M

ANSWERS:

Section 1: Intelligent Email and Password Validator.

Concept: Regular Expressions (re module)

CODE:

```
#jothika-192324235
import re
def validate_email(email):
    pattern = r"^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$"
    return re.fullmatch(pattern, email) is not None
def validate_password(password):
    pattern = r"^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$"
    return re.fullmatch(pattern, password) is not None
def validate_mobile(mobile):
    pattern = r"^[6-9]\d{9}$"
    return re.fullmatch(pattern, mobile) is not None
try:
    email = input("Enter email: ").strip()
    password = input("Enter password: ")
    mobile = input("Enter mobile number: ").strip()
```

```
    print("Valid Email" if validate_email(email)
        else "Invalid Email")
    print("Strong Password" if validate_password(password)
        else "Weak Password")
    print("Valid Mobile Number" if validate_mobile(mobile)
        else "Invalid Mobile Number")
except (EOFError, KeyboardInterrupt):
    print("\nInput cancelled.")
```

OUTPUT:

Enter email: jothika@gmail.com

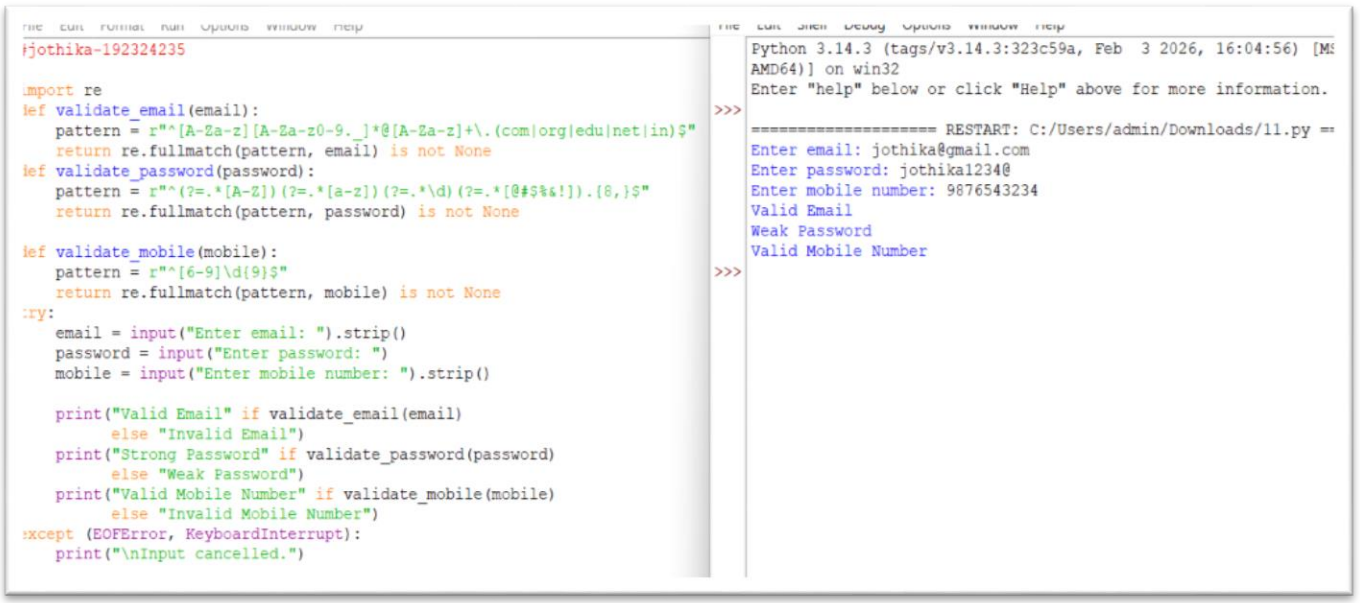
Enter password: jothika1234@

Enter mobile number: 9876543234

Valid Email

Weak Password

Valid Mobile Number



Section 2: Design a Finite State Automata (FSA) Simulator

Concept: Deterministic Finite Automaton (DFA)

This program accepts a DFA description from the user, including states, alphabet, transition table, initial state, and final states. It then tests multiple strings.

CODE:

```
#jothika-192324235

states = {
    "q0": {"a": "q1", "b": "q0"},
    "q1": {"a": "q1", "b": "q2"},
    "q2": {"a": "q1", "b": "q0"}
}

initial = "q0"

final = "q2"

while True:
    s = input("Enter string (exit to stop): ")
    if s.lower() == "exit":
        break
    state = initial
    path = [state]
    valid = True
    for ch in s:
        if ch not in ["a", "b"]:
            valid = False
            break
        state = states[state][ch]
```

```
        path.append(state)
    print("Transition Path:", " -> ".join(path))
    if valid:
        print("Accepted" if state == final else "Rejected")
    else:
        print("Invalid input")
```

OUTPUT:

Enter string (exit to stop): abaab

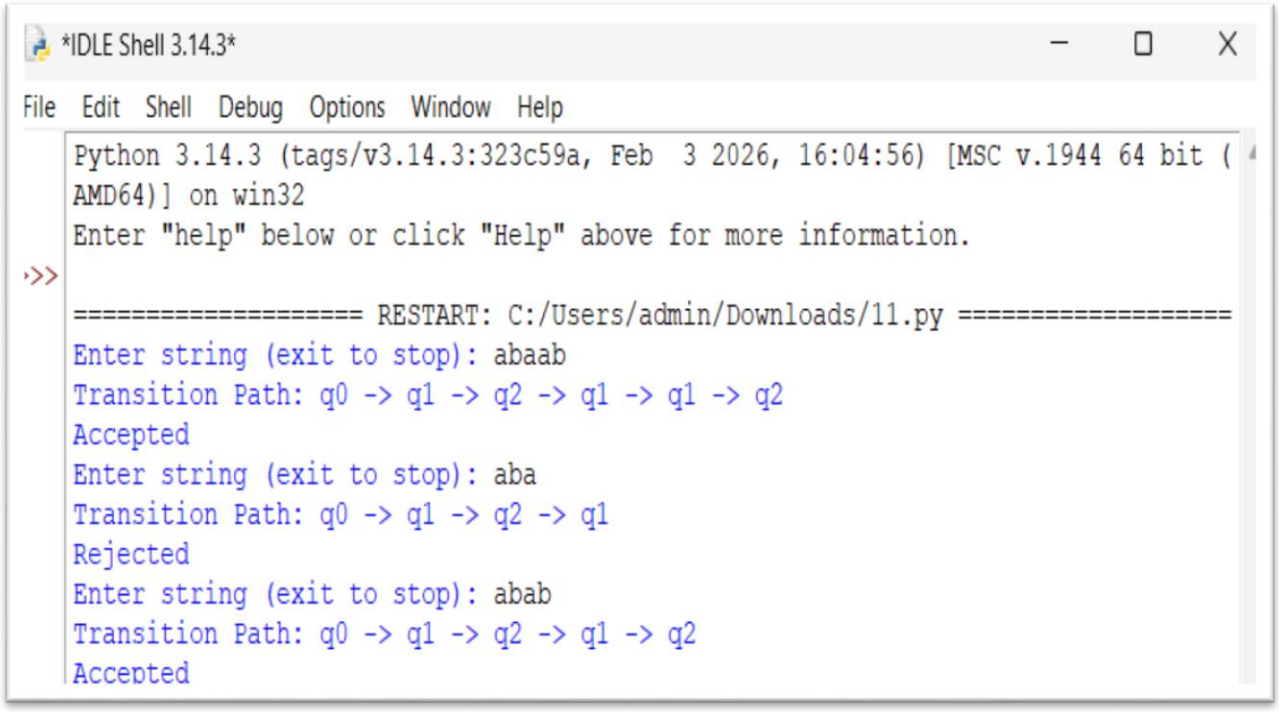
Transition Path: q0 -> q1 -> q2 -> q1 -> q1 -> q2

Accepted

Enter string (exit to stop): abab

Transition Path: q0 -> q1 -> q2 -> q1 -> q2

Accepted



Section 3: Smart Pattern Matching Engine

Concept: Advanced Regular Expressions using Python's re module.

This program searches text for dates, phone numbers, hashtags, mentions, prefixes, suffixes, and words.

CODE:

```
#jothika-192324235

import re

text = input("Enter text: ")

while True:

    print("\n1. Search Date")

    print("2. Search Phone Number")

    print("3. Search Hashtag")

    print("4. Search Mention")

    print("5. Search Prefix")

    print("6. Search Suffix")

    print("7. Search Word")

    print("8. Exit")

    choice = input("Enter choice: ")

    patterns = {

        "1": r"\b\d{2}/\d{2}/\d{4}\b",

        "2": r"\b[6-9]\d{9}\b",

        "3": r"(?<!\w)#[A-Za-z0-9_]+",

        "4": r"(?<!\w)@[A-Za-z0-9_]+",

        "5": r"\bnatural\w*",

        "6": r"\w*ing\b",
```

```
    "7": r"\bprocessing\b"
}
if choice == "8":
    break
elif choice in patterns:
    result = re.findall(patterns[choice], text, re.IGNORECASE)
    print("Matches:", result if result else "No matches found")
else:
    print("Invalid choice")
```

OUTPUT:

